

Surrogate-Assisted Modeling with Engression

Ziyi Tao, University of Michigan

Zhenyu Wang, Rutgers University

1 Introduction

This work aims to learn the conditional distribution of an outcome given covariates in the presence of surrogate data. Specifically, we observe two types of samples: a large collection of unlabeled data $\mathcal{D}_U := (X_j, Z_j)_{j=1}^N$ and a smaller set of labeled data $\mathcal{D}_L := (X_i, Z_i, Y_i)_{i=1}^n$, where X denotes the covariates, Z represents surrogate variables, and Y is the outcome of interest. In many practical applications—such as healthcare or social science studies—the labeled sample is costly to obtain, leading to an imbalanced setting with $n \ll N$. Our goal is to accurately estimate the conditional distribution of Y given X by effectively leveraging the abundant surrogate information contained in Z .

2 Methods and Evaluations

We compare two types of distribution learning procedures as follows:

- **(Baseline)** As shown in Algorithm 1, this approach uses only the labeled data $\mathcal{D}_L$ to fit an engression model of Y on X , resulting in an estimated conditional distribution denoted by $\hat{p}_{\text{base}}(y \mid x)$.

For evaluation purposes, for each fixed value of x_0 , we generate M synthetic labels $\{y^{[m]}\}_{m=1}^M$ independently from the learned distribution $\hat{p}_{\text{base}}(y \mid x = x_0)$. The generated samples are then used to compute various measures, such as quantiles, mean squared error (MSE), or other distributional metrics. The default M is set as $M = 500$.

Algorithm 1: Algorithm: Baseline

Input: Labeled Data $\{X_i, Y_i\}_{i=1}^n$

–*Training Part*–

Train an Engression model on the labeled data $\{X_i, Y_i\}_{i=1}^n$, with the learned distribution $\hat{p}_{\text{base}}(y \mid x)$

–*Evaluation Part*–

For each x_0 , generate synthetic labels $\{y^{[m]}\}_{m=1}^M \sim \hat{p}_{\text{base}}(y \mid x = x_0)$.

- **(Proposal)** As shown in Algorithm 2, this approach leverages both the unlabeled data $\mathcal{D}_U$ and the labeled data $\mathcal{D}_L$. Specifically, it first fits an engression model of Z on X using the combined data to obtain $\hat{p}(z \mid x)$. Then, based on the labeled data $\mathcal{D}_L$, it fits an engression model of Y on (X, Z) , yielding the estimated conditional distribution $\hat{p}(y \mid x, z)$.

For evaluation purposes, for each fixed value of x_0 , we generate M synthetic surrogates $\{z^{[m]}\}_{m=1}^M$ independently from the learned distribution $\hat{p}(z \mid x_0)$. For each pair $(x_0, z^{[m]})$, we then generate a corresponding synthetic label $y^{[m]}$ from the learned conditional distribution $\hat{p}(y \mid x_0, z^{[m]})$. The collection of generated labels $\{y^{[m]}\}_{m=1}^M$ is subsequently used to compute various evaluation metrics such as quantiles, mean squared error (MSE), or other distributional measures.

Algorithm 2: Algorithm: Proposal

Input: Labeled Data $\{X_i, Z_i, Y_i\}_{i=1}^n$, Unlabeled Data $\{X_i, Z_i\}_{i=1}^N$.

–*Training Part*–

Train an Engression model on data $\{X_i, Z_i\}_{i=1}^{n+N}$, with the learned distribution $\hat{p}(z \mid x)$;

Train an Engression model on the labeled data $\{X_i, Z_i, Y_i\}_{i=1}^n$, with the learned distribution $\hat{p}(y \mid x, z)$;

–*Evaluation Part*–

For each x_0 , generate synthetic surrogate $\{z^{[m]}\}_{m=1}^M \sim \hat{p}(z \mid x = x_0)$.

For each pair $(x_0, z^{[m]})$, generate a corresponding synthetic label $y^{[m]}$ from $\hat{p}(y \mid x_0, z^{[m]})$.

2.1 Evaluation Metrics

For each given x_0 , we draw synthetic labels $\{y^{[m]}\}_{m=1}^M$ using the two procedures described above. For comparison against the ground truth, we also have access to an oracle sampler from the true conditional law $p(y|x = x_0)$, from which we generate $\{y_{\text{orac}}^{[r]}\}_{r=1}^R$ (with $R = 2000$ as the default).

We report the following summary statistics for each x_0 . Note that these quantities can be averaged over a test set $\mathcal{X}_0$ of covariate values of an overall score.

- MSE: it computes

$$\left(\frac{1}{M} \sum_{m=1}^M y^{[m]} - \frac{1}{R} \sum_{r=1}^R y_{\text{orac}}^{[r]} \right)^2.$$

- Quantile error. For a target quantile level $\tau \in (0, 1)$, the pinball loss is $\rho_\tau(a) = a \cdot (\tau - \mathbf{1}\{a < 0\})$. Let $\hat{q}_\tau(x_0)$ be the τ -quantile of $\{y^{[m]}\}_{m=1}^M$ and $q_{\tau, \text{orac}}(x_0)$ be that of $\{y_{\text{orac}}^{[r]}\}_{r=1}^R$. We report

$$\rho_\tau(\hat{q}_\tau(x_0) - q_{\tau, \text{orac}}(x_0)).$$

We set $\tau = 0.1, 0.25, 0.5, 0.75, 0.9$ as default.

- Total Variation Distance. For kernel density estimator $\hat{p}, \hat{q}_1$ and $\hat{q}_2$ calculated based on the real sample $Y_{i, \text{True}}$ from the population, predicted sample from $\text{model}_1 : Y_{i, \text{Pred}, 1}$ and predicted sample from $\text{model}_2 : Y_{i, \text{Pred}, 2}$. Total Variation Distance (TVD) is calculated by extracting values of density from the uniform grid with spacing s .

$$\text{TVD}_k(i) \approx \frac{1}{2} \sum_j |\hat{p}(\text{grid}_j | X_i) - \hat{q}_k(\text{grid}_j | X_i)| \times s, k = 1, 2$$

$$\text{Mean}(\text{TVD}_k) = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \text{TVD}_k(i)$$

- KL Divergence.
- Wasserstein Distance. For each X_i , 1-d Wasserstein distance between real distribution and estimated distribution takes the form of

$$W_k(i) = W_1(Y_{i, \text{True}}, Y_{i, \text{Pred}, k}) = \int |F_P(y | X_i) - F_{Q_k}(y | X_i)| dy, k = 1, 2$$

$$\bar{W}^{(k)} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} W_k(i)$$

where k represents model index $\in 1, 2$.

3 Simulations

We construct a synthetic data-generating mechanism following the pathway:

$$X \rightarrow Z \rightarrow Y$$

The variables are generated as follows: $X = (X_1, X_2, X_3)$, $X_i \stackrel{\text{i.i.d}}{\sim} \text{Unif}([-5, 5])$,

$$Z_{\text{clean}} = f_Z(X), \quad \text{and} \quad Z = Z_{\text{clean}} + \epsilon_z, \quad \text{with} \quad \epsilon_z \sim N(0, 0.2)$$

$$Y = f_Y(Z_{\text{clean}}) + \epsilon_y = f_Y(f_Z(X)) + \epsilon_y, \quad \text{with} \quad \epsilon_y \sim N(0, 0.5)$$

The structural functions $f_Z(\cdot), f_Y(\cdot)$ are defined by

$$f_Z(X) = 2\sin(X_1) + 0.05X_2^2 - 0.1X_3,$$

and

$$f_Y(Z_{\text{clean}}) = \begin{cases} 0.1Z_{\text{clean}}^3 & z < 0 \\ \log(1 + \exp(z_{\text{clean}})) + 0.5\sin(Z_{\text{clean}}) & 0 \leq z < 1 \\ \sqrt{|Z_{\text{clean}}|} + 2 & z \geq 1 \end{cases}$$

Conditional on the remaining coordinates $X_{2:p}$, the functional dependence of Y on X_1 exhibits piecewise nonlinearity, as illustrated in Figure 1.

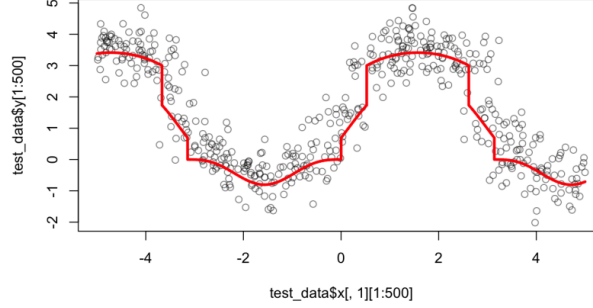


Figure 1: True conditional relationship between Y and X_1 given $X_{2:p}$. The red curve represents the deterministic component $f_Y(f_Z(X))$.

We consider the varying sample size regime, with (i) ($n = 100, N = 400$); (ii) ($n = 200, N = 800$); (iii) ($n = 400, N = 1600$).

Training Sample	Model	Conditional Mean	Quantile			Distribution	
		MSE	0.25 loss	0.5 loss	0.75 loss	mean TVD	mean Wasserstein
500(100 labeled + 400 unlabeled)	x-y	1.541(1.110)	0.344(0.118)	0.477(0.196)	0.387(0.158)	0.620	1.527
	x-z-y	0.461(0.802)	0.242(0.104)	0.319(0.176)	0.260(0.147)	0.292	0.359
1000(200 labeled + 800 unlabeled)	x-y	1.100(1.098)	0.298(0.109)	0.411(0.192)	0.330(0.149)	0.339	0.511
	x-z-y	0.415(0.790)	0.227(0.095)	0.295(0.158)	0.238(0.129)	0.267	0.399
2000(400 labeled + 1600 unlabeled)	x-y	0.770(0.968)	0.253(0.090)	0.338(0.154)	0.275(0.126)	0.508	1.036
	x-z-y	0.205(0.486)	0.204(0.070)	0.258(0.114)	0.208(0.098)	0.193	0.242

Figure 2: Summary Result

Conditional mean For each test data j , we conduct n simulations to estimate the conditional mean and calculate the distance between the average of estimated conditional mean and true mean.

$$d_j := \left| \frac{1}{n} \sum_{i=1}^n \hat{\mu}_{i,j} - \mu_j \right|, j = 1, 2, \dots, N$$

where n represents the number of simulations, N represents the size of test data.

Training Sample	Model	Mean d
500(100 labeled + 400 unlabeled)	x-y	0.870
	x-z-y	0.343
1000(200 labeled + 800 unlabeled)	x-y	0.666
	x-z-y	0.302
2000(400 labeled + 1600 unlabeled)	x-y	0.508
	x-z-y	0.217

Figure 3: Mean Distance

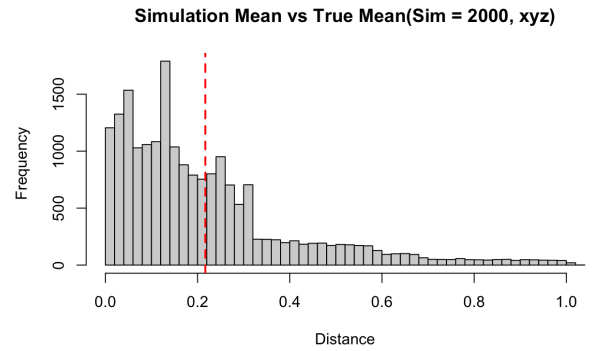
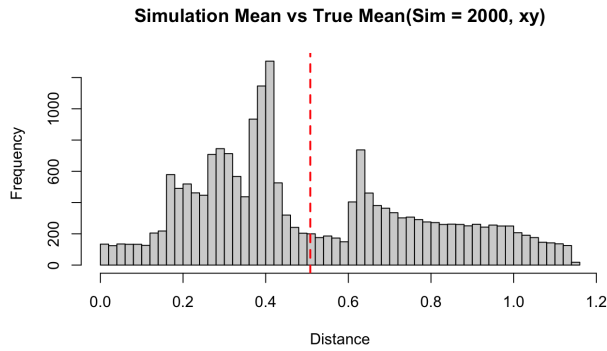
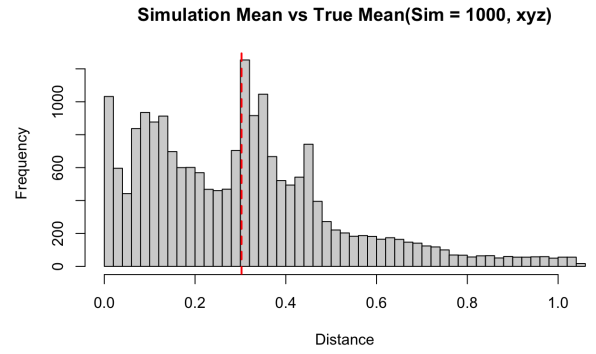
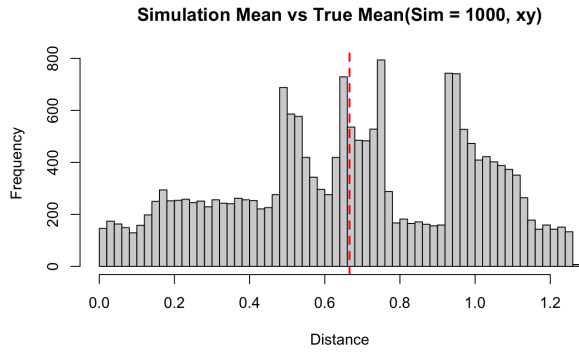
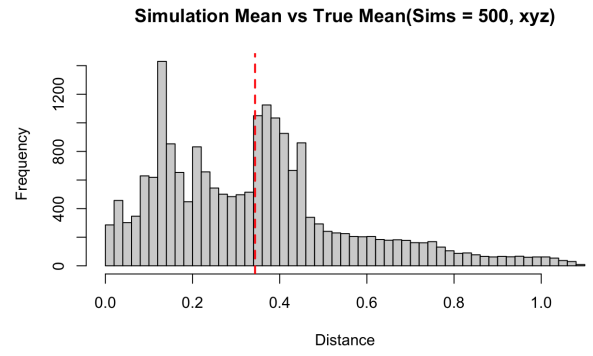
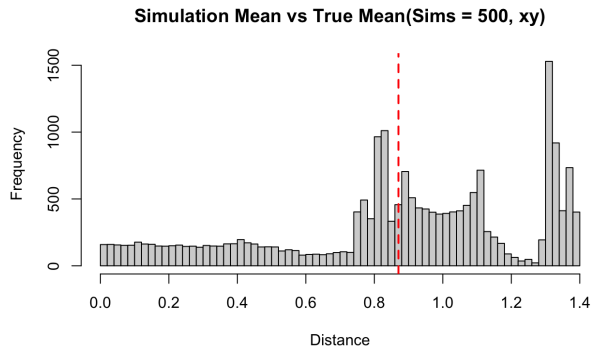


Figure 6: Overall figure caption describing both subfigures.

4 Previous by Ziyi

2 Data Split

- Size of Testset: 20000
- Training Set:
 - Dataset 1: 200 labeled, 1800 unlabeled
 - Dataset 2: 2000 labeled, 18000 unlabeled

- Dataset 3: 20000 labeled, 180000 unlabeled

Labeled data contain $\{X, y, z\}$, unlabeled data contain $\{X, z\}$.

3 Statistics Calculation

X-y Path For each X, generate 800 y based on engression model $X \rightarrow y$. Calculate sample mean, quantile and distribution statistics.

X-z-y Path

- Step 1: For each X, generate 800 z based on engression model $X \rightarrow z$, i.e. $\hat{p}(Z|X)$.
- Step 2: For each set $\{X, z\}$, generate 1 y based on engression model $X, z \rightarrow y$, i.e., $\hat{p}(y|X, Z)$.
- Step 3: Consider $\{y_i\}$ as a bunch of data points drawn from $\hat{p}(y|X)$, calculated sample mean and quantile.

4 Model Fit

Consider the case where the training set size is 2000.

Algorithm 3: Approach 0: Baseline, $X \rightarrow y$

Input: 200 labeled observations $\{X, y\}$

Output: Engression model $\hat{f}_0 : X \rightarrow y$

Step 1: Train an Engression model on the 200 labeled samples.

Step 2: Predict y directly from X .

- Prediction Path: Use the trained Engression model $\hat{f}_0$ to estimated y based on X.

$$\hat{y} = \hat{f}_0(X)$$

Algorithm 4: Approach 1: $x \rightarrow y$ with imputation

Input: 200 labeled observations $\{X, y\}$

Output: Engression model $\hat{f}_1 : X \rightarrow y$

Step 1: Train an Engression model f_{xy} on the 200 labeled samples to predict y based on X.

Step 2: Impute missing y values for the 1800 unlabeled samples using f_{xy} .

Step 3: Train the final Engression model $\hat{f}_1$ using the imputed dataset $X \rightarrow y$.

- Prediction Path: Use the trained Engression model $\hat{f}_1$ to estimated y based on X.

$$\hat{y} = \hat{f}_1(X)$$

Algorithm 5: Approach 2: $X, z \rightarrow y$, no imputation

Input: 200 labeled observations $\{X, y\}$

Output: Engression model $\hat{f}_{2a} : X, z \rightarrow y$, $\hat{f}_{2b} : X \rightarrow z$

Step 1: Train an Engression model $\hat{f}_{2a}$ on the 200 labeled samples to predict y based on X, z ($X, z \rightarrow y$).

Step 2: Train another Engression model $\hat{f}_{2b}$ on the 2000 samples with X, z to predict z based on X ($X \rightarrow z$).

- Prediction path: For a given X , first predict

$$\hat{z} = \hat{f}_{2b}$$

then predict

$$\hat{y} = \hat{f}_{2a}(X, \hat{z})$$

Algorithm 6: Approach 3: $X, z \rightarrow y$ with imputation

Input: 200 labeled samples $\{X, z, y\}$, 1800 unlabeled samples $\{X, z\}$

Output: Final Engression models $\hat{f}_{3b} : (X, z) \rightarrow y$, $\hat{f}_{3a} : X \rightarrow z$

Step 1: Train an Engression model $\hat{f}$ on the 200 labeled data to predict y from (X, z) .
 $(X, z) \rightarrow y$

Step 2: Train an Engression model $\hat{f}_{3a}$ on all 2000 training data to predict z from X .
 $X \rightarrow z$

Step 3: Impute missing y values for the 1800 unlabeled samples:

For each $\{X, z\}$, predict 5 possible y 's using $\hat{f}$.

This yields a total of $1800 \times 5 + 200 = 9200$ augmented samples $\{X, z, y\}$.

Step 4: Train the final Engression model $\hat{f}_{3b}$ using the imputed dataset.
 $(X, z) \rightarrow y$

- Prediction path: For a given X , first predict

$$\hat{z} = \hat{f}_{3b}$$

then predict

$$\hat{y} = \hat{f}_{3a}(X, \hat{z})$$

5 Results

5.0 Model Comparison

- Estimated conditional mean
- Estimated conditional quantile (0.25, 0.5, 0.75)
- Estimated conditional distribution (mean Total Variance Distance, mean Wasserstein distance)

5.1 General Performance Rank(best to worst):

- $X \rightarrow z \rightarrow y$ (with imputation)
- $X \rightarrow z \rightarrow y$ (no imputation) **similar to the previous model**
- $X \rightarrow y$ (no imputation)
- $X \rightarrow y$ (with imputation)

Training Sample	Model	Conditional Mean	Quantile			Distribution	
		MSE	0.25 loss	0.5 loss	0.75 loss	mean TVD	mean Wasserstein
2000(200 labeled + 1800 unlabeled)	Baseline(xy_noimp)	0.3245	0.2070	0.2862	0.2054	0.324	0.484
	x-y	0.2504	0.4826	0.7543	0.5938	0.331	0.496
	x-z-y(no imp)	0.1132	0.1921	0.2225	0.1883	0.243	0.304
	x-z-y	0.1004	0.1879	0.2254	0.1802	0.237	0.275
20000(2000 labeled + 18000 unlabeled)	Baseline(xy_noimp)	0.1763	0.1917	0.2437	0.2213	0.357	0.578
	x-y	0.2124	0.2117	0.2734	0.2222	0.310	0.455
	x-z-y(no imp)	0.0540	0.1792	0.2178	0.1777	0.173	0.222
	x-z-y	0.0564	0.1745	0.2232	0.1765	0.172	0.221
200000(20 labeled + 180 unlabeled)	Baseline(xy_noimp)	0.1472	0.1858	0.2534	0.1908	0.210	0.283
	x-y	0.1833	0.2047	0.2546	0.1957	0.227	0.315
	x-z-y(no imp)	0.0472	0.1755	0.2183	0.1748	0.148	0.188
	x-z-y	0.0445	0.173	0.2190	0.1757	0.168	0.218

Figure 7: General Results

5.2 Conditional Mean

- Using prediction path normally works better than directly predicting y from X.
- Imputed models showed no apparent improvement in terms of conditional mean.

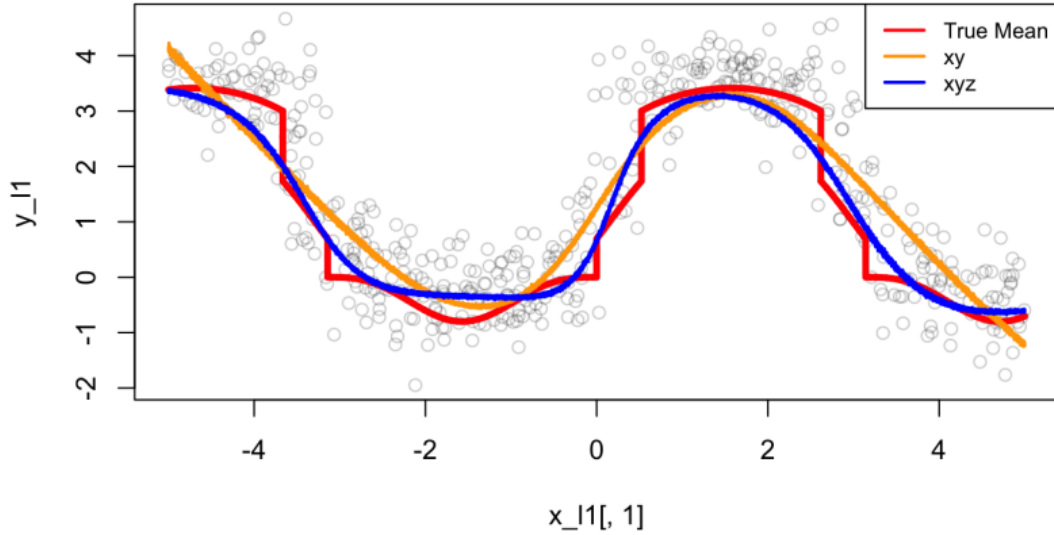


Figure 8: Sample 2000 Mean

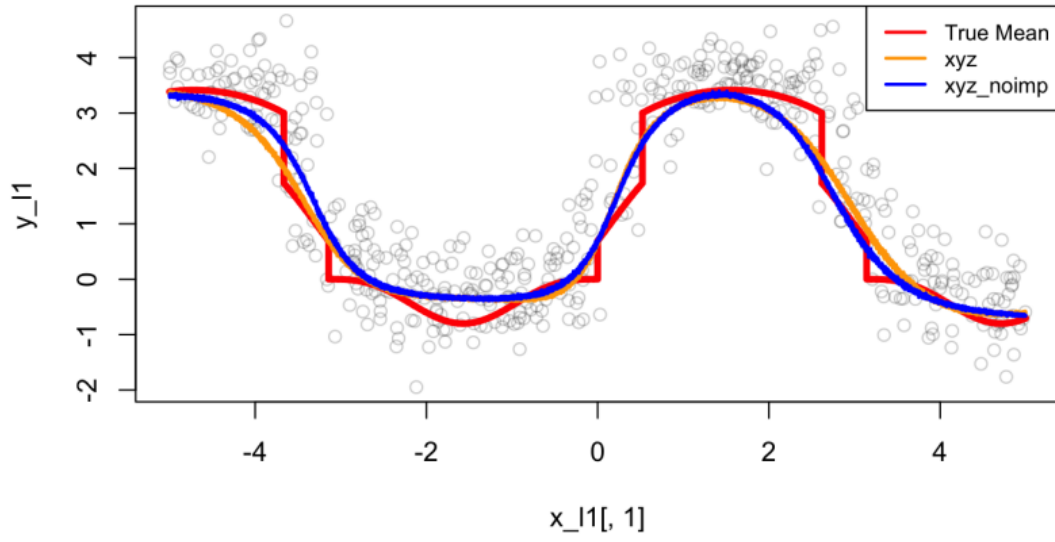


Figure 9: Enter Caption

5.3 Quantile

- **xy path with imputation** does not behave very stable when sample size is small, may bring some negative effect than xy path without imputation

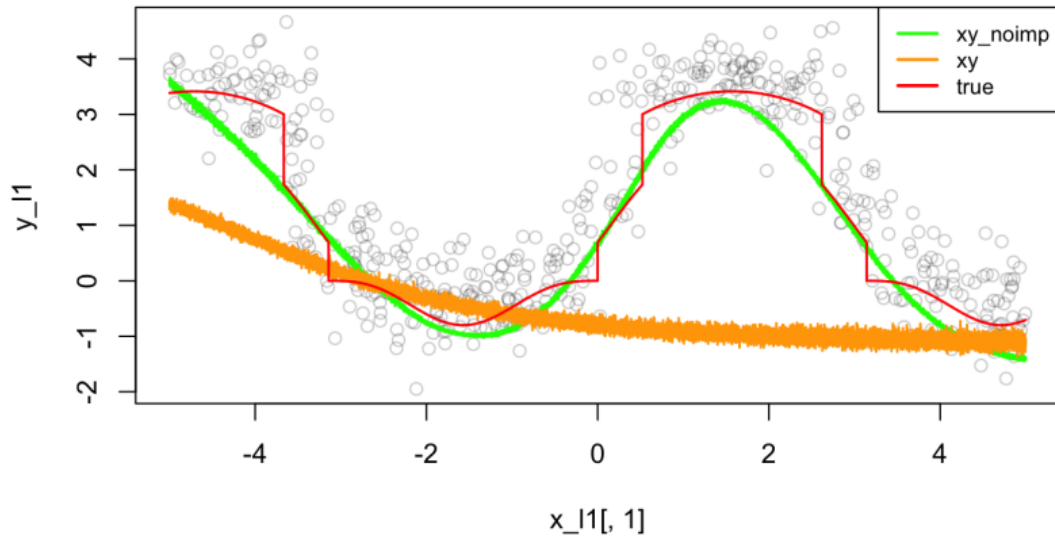


Figure 10: Sample size = 2000, quantile 0.25

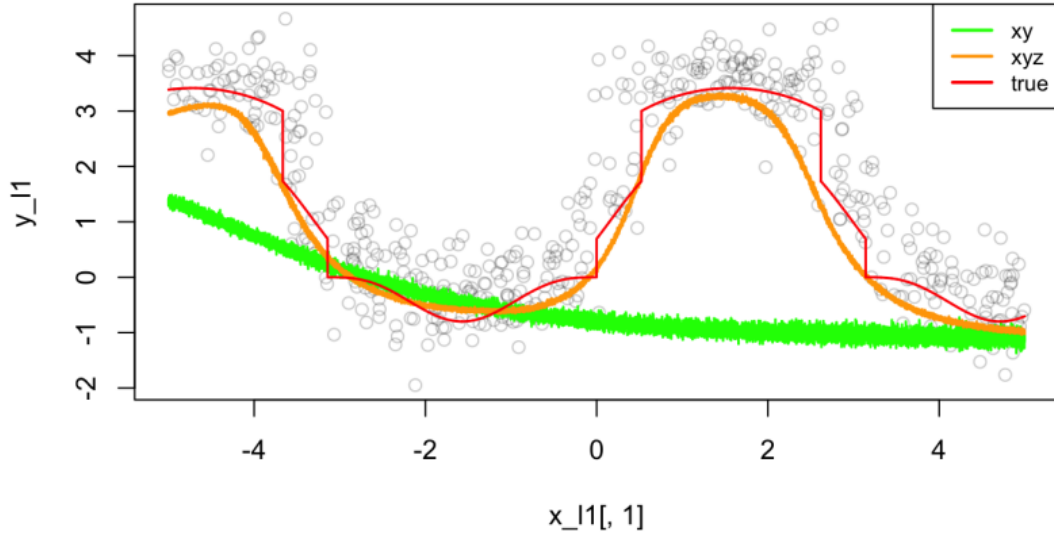


Figure 11: Sample size = 2000, quantile 0.25

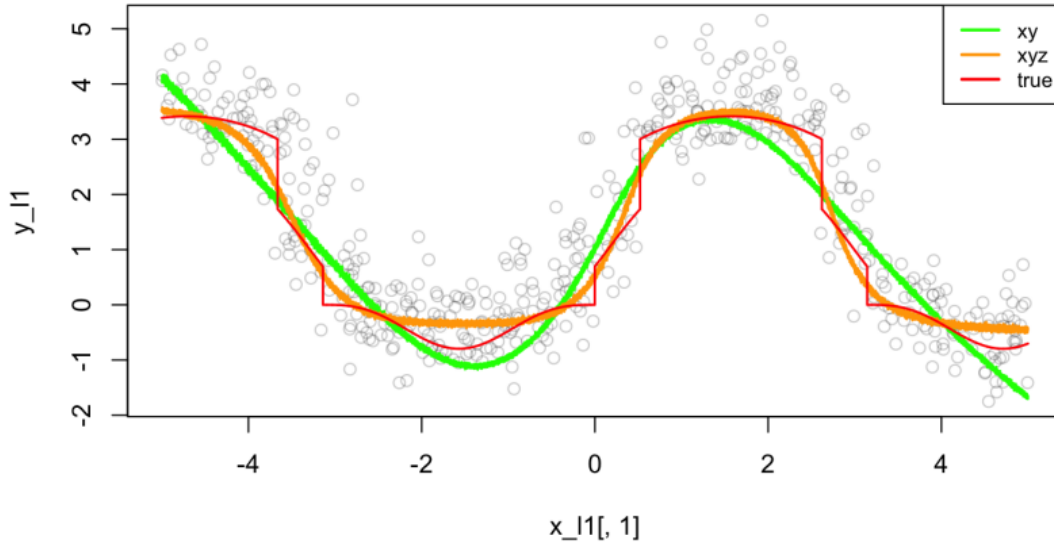


Figure 12: Sample size = 20000, quantile 0.25

5.3 Distribution Similar performance to conditional mean and quantile prediction.

Distribution Methodology **model1:x-z-y, model2: x-y**

- Step 1: Generate real samples

For every observation in testX, use true generation function generate M samples of Y, get real testY with size $n_{test} \times M$:

$$Y_{i,True} = \{Y_{i,1}, Y_{i,2}, \dots, Y_{i,M}\} \sim P(y|X_i)$$

- Step 2: Generate Predicted Samples

Using the methods mentioned in Section 3, we got 2 predicted sample:

$$Y_{i,Pred,1} \sim Q_1(y|X_i)$$

$$Y_{i,Pred,2} \sim Q_2(y|X_i)$$

- Step 3: Calculate Mean TVD and Wasserstein distance

For each X_i , 1-d Wasserstein distance between real distribution and estimated distribution(use transport::wasserstein1d()):

$$W_k(i) = W_1(Y_{i,True}, Y_{i,Pred,k}) = \int |F_P(y|X_i) - F_{Q_k}(y|X_i)| dy, k = 1, 2$$

$$Mean(W_{1,modelk}) = \frac{1}{n_{test}} \sum_{i=1}^{n_{test}} W_k(i)$$

For mean TVD, $\hat{p}, \hat{q}_1, \hat{q}_2$ are KDE based respectively on real sample $Y_{i,True}$, predicted sample from model1 $Y_{i,Pred,1}$, predicted sample from $Y_{i,Pred,2}$. The code calculates the Total Variation Distance (TVD) by extracting the density values from these Kernel Density Estimation (KDE) objects at the points on the uniform grid (grid). It then multiplies the absolute difference in these density values by the grid step size (step) to approximate the integral, which is the definition of the TVD.

$$TVD_k(i) \approx \frac{1}{2} \sum_j |\hat{p}(grid_j|X_i) - \hat{q}_k(grid_j|X_i)| \times step, k = 1, 2$$

$$Mean(TVD_k) = \frac{1}{n_{test}} \sum_{i=1}^{n_{test}} TVD_k(i)$$

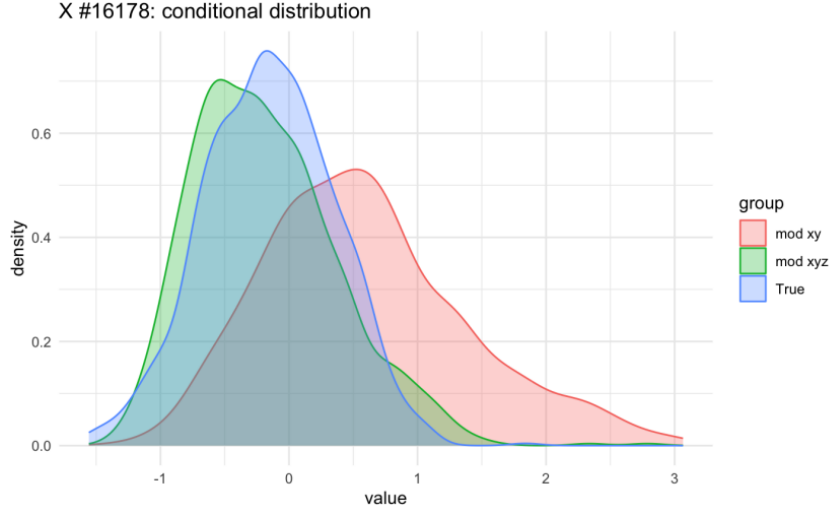


Figure 13: Sample size = 2000

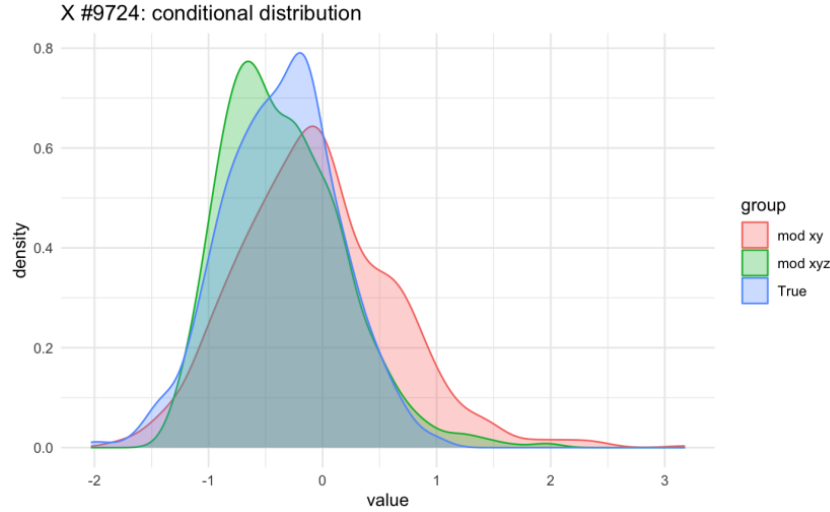


Figure 14: Sample size = 2000

6 Discussion

6.1 General results Rank(best to worst):

- $X \rightarrow z \rightarrow y$ (with imputation)
- $X \rightarrow z \rightarrow y$ (no imputation) **similar to the previous model**
- $X \rightarrow y$ (no imputation)
- $X \rightarrow y$ (with imputation)

Using prediction path normally works better than directly predicting y from X .

Imputed models showed no apparent improvement.

xy path with imputation does not behave very stable when sample size is small, may bring some negative effect than xy path without imputation.

6.2 More Imputation seems to influence the prediction power for Xy models negatively while shows no apparent influence on Xyz models.

- We use an engression trained on limited labeled $X \rightarrow y$ data for imputation, if this model is biased, the imputed y 's will not reflect the true conditional distribution and will introduce more error than expected.
- When X, y relationship is complicated, the benefit of sample enlargement brought by imputation may be reduced by the error and noise brought by a less accurate imputation model.
- In $X \rightarrow z \rightarrow y$ path, the noise brought by imputation model may be balanced out by the relative accurate imputation model $X \rightarrow z$ compared to $X \rightarrow y$, which is easier to predict. In order for imputation to show some more positive improvement to model performance, more accurate prediction model may be needed(deeper network, more nodes...)